

PKCERT AI & Software Development Internship

Task 09 – Support Vector Machines (SVM) & k-Nearest Neighbors (kNN)

Dataset: Titanic Passenger Survival Dataset

Prepared by: Abdul Rehman

Submission Date: 15/7/26

Part A – Dataset Selection & Preparation

A.1 Dataset Description

The Titanic dataset is a widely used public benchmark dataset for binary classification. It records information about passengers aboard the RMS Titanic and whether each passenger survived the 1912 sinking. It was selected because it is a moderately sized, mixed-type (numerical + categorical) tabular dataset with a clearly defined binary target, making it well suited for comparing SVM and kNN classifiers.

Target Variable: survived — 0 = Did not survive, 1 = Survived

Features used:

Feature	Description	Type
pclass	Ticket/passenger class (1st, 2nd, 3rd)	Categorical (ordinal)
sex	Passenger gender	Categorical
age	Age in years	Numerical
sibsp	# siblings/spouses aboard	Numerical
parch	# parents/children aboard	Numerical
fare	Ticket fare paid	Numerical
embarked	Port of embarkation (C/Q/S)	Categorical
family_size	Engineered: sibsp + parch + 1	Numerical

A.2 Data Preprocessing

The raw dataset (891 rows, 15 columns) contained missing values in age (177), embarked (2), deck (688), and embark_town (2). The following preprocessing steps were applied:

- Selected 8 relevant columns and dropped high-missing / redundant columns (deck, embark_town, class, who, alive, adult_male, alone).
- Imputed missing age values with the column median (robust to outliers/skew).
- Imputed missing embarked values with the column mode.
- Engineered a new feature, family_size = sibsp + parch + 1, to capture total family group size.
- Label-encoded sex (male = 1, female = 0).
- One-hot encoded embarked (drop-first to avoid multicollinearity).
- Scaled all features using StandardScaler (mean = 0, std = 1) — essential for SVM and kNN, since both are distance/margin-based algorithms sensitive to feature magnitude.

After preprocessing, the dataset had 891 rows × 10 columns with zero missing values.

A.3 Train-Test Split

The data was split 80/20 into training and test sets using stratified sampling on the target variable to preserve class balance in both sets.

Split	Rows	Class 0 (Did not survive)	Class 1 (Survived)
Full dataset	891	549 (61.6%)	342 (38.4%)
Training set	712	~439	~273
Test set	179	110	69

Part B – Support Vector Machine

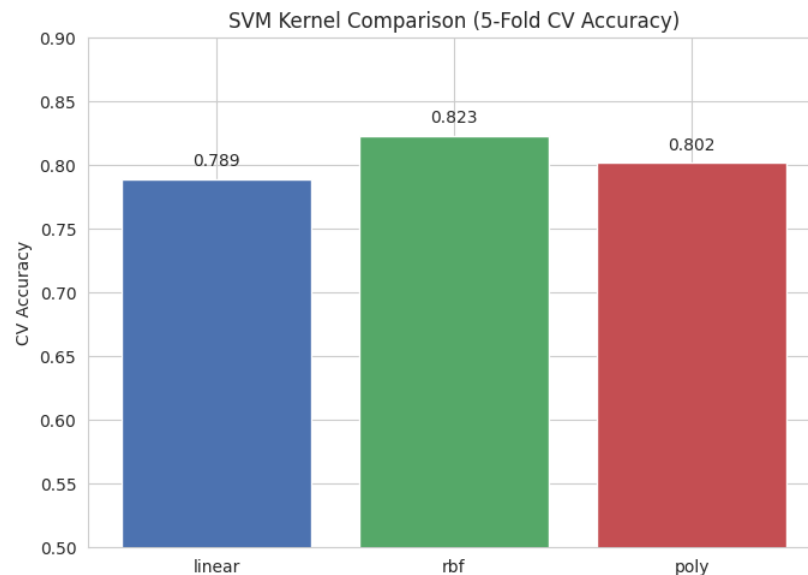
B.1 What is SVM?

A Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane which maximally separates classes in feature space. The training points closest to this boundary — called support vectors — determine its position and orientation. For data that is not linearly separable, SVM applies the kernel trick to implicitly project data into a higher-dimensional space where a linear separator can be found.

B.2 Kernel Selection

Three kernels were compared using 5-fold cross-validation on the training set:

Kernel	CV Accuracy	Std Dev
Linear	0.7894	± 0.0155
RBF	0.8231	± 0.0099
Polynomial	0.8020	± 0.0318



The RBF (Radial Basis Function) kernel achieved the highest and most stable cross-validated accuracy, and was therefore selected. This is consistent with domain expectations: Titanic survival depends on non-linear interactions between features (e.g. class, sex, and age jointly determine survival likelihood in ways a straight-line boundary cannot capture), and RBF models these interactions without the higher variance risk associated with a high-degree polynomial kernel.

B.3 Hyperparameter Tuning

GridSearchCV (5-fold) was used to tune the RBF kernel's C (regularization strength) and gamma (kernel coefficient) parameters.

Best parameters found: C = 10, gamma = 0.1

Best cross-validated accuracy: 0.8273

B.4 Evaluation on Test Set

Metric	Score
Accuracy	0.8101

Metric	Score
Precision	0.8070
Recall	0.6667
F1-Score	0.7302

Confusion Matrix (rows = actual, columns = predicted):

	Predicted: Died	Predicted: Survived
Actual: Died	99	11
Actual: Survived	23	46

The model correctly identified 99 of 110 non-survivors and 46 of 69 survivors. Precision (0.807) indicates that when the model predicts survival, it is correct about 81% of the time. Recall (0.667) shows it misses roughly a third of true survivors — reflecting the harder-to-detect minority class in an imbalanced dataset.

B.5 SVM: Advantages, Limitations & Applications

Advantages

- Effective in high-dimensional spaces, even when features outnumber samples.
- Memory-efficient, since the decision function depends only on support vectors.
- The kernel trick enables flexible, non-linear decision boundaries.
- Robust to overfitting when the margin is properly regularized via C .

Limitations

- Training scales poorly to very large datasets (roughly $O(n^2)$ – $O(n^3)$).
- Performance is sensitive to kernel choice and hyperparameter tuning (C , γ).
- Does not directly output class probabilities without extra calibration (e.g. Platt scaling).
- Less interpretable than simpler models such as logistic regression or decision trees.

Applications

- Text and document classification, e.g. spam detection.
- Image classification and face detection.
- Bioinformatics, e.g. gene/protein classification, cancer diagnosis.
- Handwriting and digit recognition.

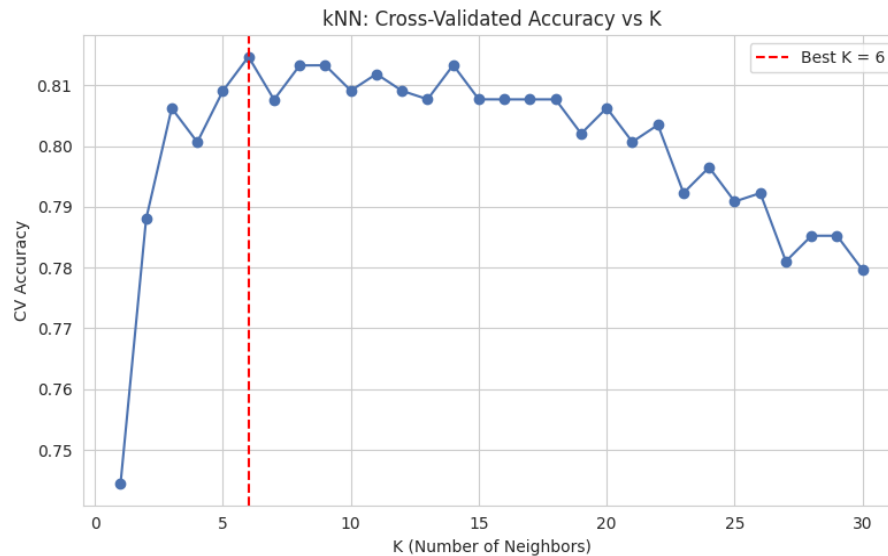
Part C – k-Nearest Neighbors

C.1 What is kNN?

k-Nearest Neighbors (kNN) is a non-parametric, instance-based learning algorithm. To classify a new data point, it identifies the K closest points in the training data (typically by Euclidean distance) and assigns the majority class among them. It performs no explicit training phase — often called 'lazy learning' — since all computation happens at prediction time.

C.2 Choosing K

K values from 1 to 30 were evaluated using 5-fold cross-validation on the training set (the elbow method).



Best K found: 6 (Cross-validated accuracy = 0.8147)

A very small K (e.g. K = 1) tends to overfit, closely following noise in the training data. A very large K oversmooths the decision boundary and can underfit. K = 6 was selected as it lies at the point where cross-validated accuracy peaks and stabilizes, giving a reasonable bias-variance balance.

C.3 Evaluation on Test Set

Metric	Score
Accuracy	0.8156
Precision	0.8333
Recall	0.6522
F1-Score	0.7317

Confusion Matrix (rows = actual, columns = predicted):

	Predicted: Died	Predicted: Survived
Actual: Died	101	9
Actual: Survived	24	45

kNN (K=6) correctly identified 101 of 110 non-survivors and 45 of 69 survivors. It achieved slightly higher precision than SVM (0.833 vs 0.807) but marginally lower recall (0.652 vs 0.667), resulting in a near-identical overall F1-score.

C.4 kNN: Advantages, Limitations & Applications

Advantages

- Simple to understand and implement; no explicit training phase.
- Naturally handles multi-class classification problems.
- Non-parametric — makes no assumptions about the underlying data distribution.
- Decision boundary can adapt flexibly to complex, local patterns in the data.

Limitations

- Prediction is slow on large datasets, since distance to all training points must be computed.
- Sensitive to feature scaling and to irrelevant or noisy features.
- Susceptible to the curse of dimensionality — performance degrades as feature count grows.
- Sensitive to class imbalance and to the specific choice of K.

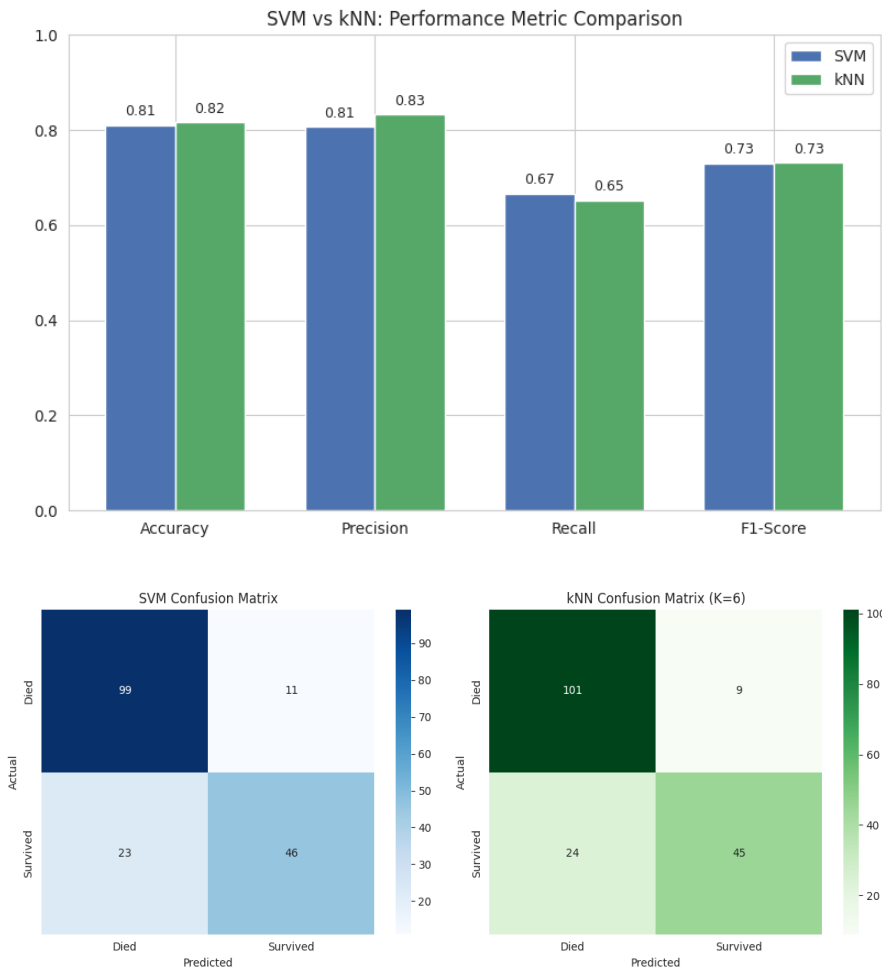
Applications

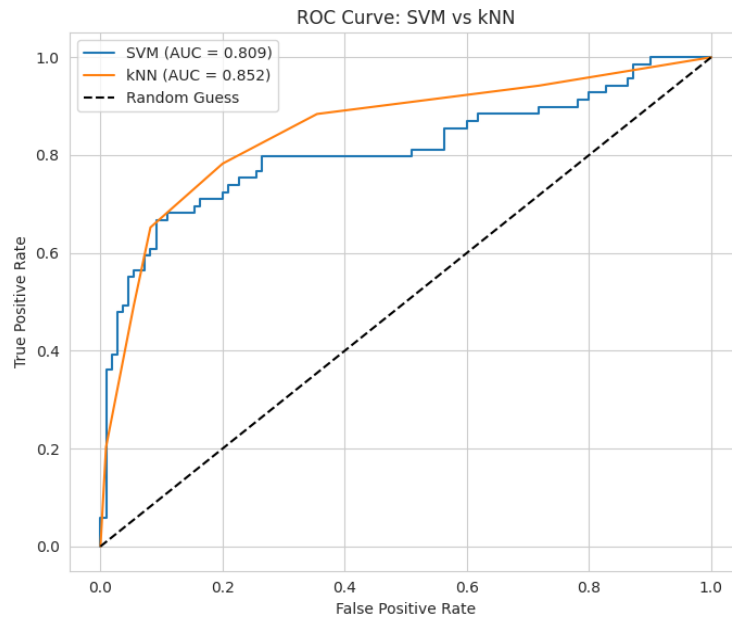
- Recommender systems (e.g. collaborative filtering).
- Pattern recognition, handwriting and OCR.
- Anomaly and fraud detection.
- Credit scoring and medical diagnosis support systems.

Part D – Comparative Analysis

D.1 Side-by-Side Performance

Model	Accuracy	Precision	Recall	F1-Score
SVM (RBF, C=10, gamma=0.1)	0.8101	0.8070	0.6667	0.7302
kNN (K=6)	0.8156	0.8333	0.6522	0.7317





D.2 Discussion

SVM (RBF) and kNN (K=6) achieve near-identical performance on the Titanic test set, with F1-scores differing by less than 0.002 (0.7302 vs 0.7317). This is a common outcome on small-to-medium tabular datasets with a moderate number of well-engineered, scaled features: both models are capable of tracing similar non-linear decision boundaries once the feature space is standardized.

- SVM's margin-maximizing objective makes it somewhat more robust to a small number of noisy or borderline points, and it generalizes slightly better on datasets with a small training-set-to-feature ratio.
- kNN is more sensitive to local data density and to class imbalance; both models show reduced recall on the minority 'survived' class (~62% did-not-survive vs ~38% survived in this dataset), which is expected without explicit class-imbalance handling (e.g. class weighting or resampling).
- Computationally, SVM has a heavier training/tuning cost (particularly during grid search over C and gamma) but very fast prediction at inference time. kNN has essentially no training cost but slower prediction, since it must compute distances to the entire training set for every new query.

D.3 Recommendation

Based on F1-score, kNN (K=6) scores marginally higher than SVM (RBF) on this test split (0.7317 vs 0.7302) — a difference small enough to be within normal run-to-run variance and not practically significant. Given that the two models are statistically comparable in performance, the recommendation depends on secondary priorities:

- If prediction speed and simplicity are priorities (e.g. deploying to a lightweight service), kNN (K=6) is a reasonable, easy-to-maintain choice.
- If robustness to noise, a smaller memory footprint at inference time, and better theoretical generalization guarantees are priorities, SVM (RBF) is the stronger choice for a production setting.

Overall recommendation: SVM (RBF) is recommended as the primary model for this dataset due to its more principled generalization behavior and efficient inference, with kNN retained as a simple, well-performing baseline for comparison.

Conclusion

Both SVM and kNN were successfully implemented, tuned, and evaluated on the Titanic dataset following a complete preprocessing pipeline covering missing-value imputation, categorical encoding, feature engineering, and feature scaling. Cross-validation was used throughout — to select the SVM kernel and hyperparameters, and to select the optimal K for kNN — ensuring both models were fairly tuned before final comparison on the held-out test set. The near-identical performance of the two models on this dataset illustrates a broader lesson in applied machine learning: when accuracy is comparable, model choice often comes down to secondary factors such as dataset size, latency requirements, interpretability, and deployment constraints rather than raw predictive performance alone.